

FAANG Interview FlashCards

A reorganized collection of essential formulas, patterns, and code snippets for coding interviews.
Organized from **Easy** → **Medium** → **Hard**.

How to Use: Each FlashCard contains the concept, implementation, and a **FAANG-level practice problem** with solution and key insight. Practice these to master each pattern!

Table of Contents

Level 1: Foundations (Easy)

- [FlashCard 1.1: Array Index Formulas](#)
- [FlashCard 1.2: 2D to 1D Array Mapping](#)
- [FlashCard 1.3: Sub-Grid Index \(Sudoku Pattern\)](#)
- [FlashCard 1.4: Basic Math Formulas](#)
- [FlashCard 1.5: Prime Number Check](#)
- [FlashCard 1.6: Basic String Operations](#)
- [FlashCard 1.7: Basic Bit Operations](#)
- [FlashCard 1.8: Sum/Product of Array](#)

Level 2: Simple Algorithms (Easy-Medium)

- [FlashCard 2.1: Two Pointers - Remove Duplicates](#)
- [FlashCard 2.2: Two Pointers - Container With Most Water](#)
- [FlashCard 2.3: Binary Search - Basic](#)
- [FlashCard 2.4: Binary Search in Rotated Array](#)
- [FlashCard 2.5: Sliding Window - Fixed Size](#)
- [FlashCard 2.6: Sliding Window - Minimum Window Substring](#)
- [FlashCard 2.7: Longest Substring Without Repeating](#)
- [FlashCard 2.8: Merge Sort](#)
- [FlashCard 2.9: Quick Sort](#)
- [FlashCard 2.10: Linked List - Reverse](#)
- [FlashCard 2.11: Detect Cycle in Linked List](#)

Level 3: Data Structures (Medium)

- [FlashCard 3.1: Tree - Node Depth & Height](#)
- [FlashCard 3.2: Tree - Node Count](#)
- [FlashCard 3.3: Valid Parentheses](#)
- [FlashCard 3.4: Largest Rectangle in Histogram](#)
- [FlashCard 3.5: Number of Islands](#)
- [FlashCard 3.6: BFS Shortest Path](#)
- [FlashCard 3.7: Union-Find \(DSU\)](#)

Level 4: Dynamic Programming (Medium-Hard)

- [FlashCard 4.1: Fibonacci](#)
- [FlashCard 4.2: Climbing Stairs](#)
- [FlashCard 4.3: House Robber](#)
- [FlashCard 4.4: Longest Common Subsequence](#)
- [FlashCard 4.5: Edit Distance](#)

Level 5: Advanced Topics (Hard)

- [FlashCard 5.1: KMP String Matching](#)
- [FlashCard 5.2: Rabin-Karp](#)
- [FlashCard 5.3: Sieve of Eratosthenes](#)
- [FlashCard 5.4: Find All Divisors](#)
- [FlashCard 5.5: Min/Max Heap](#)
- [FlashCard 5.6: Power with Modulo](#)
- [FlashCard 5.7: Geometry - Distance Formulas](#)

Levels Overview

Level	Topics	Difficulty
1	Array Index, Math, String, Bit Ops	Easy
2	Two Pointers, Binary Search, Sorting, Linked List	Easy-Medium
3	Trees, Stacks, Graphs, Union-Find	Medium

Level	Topics	Difficulty
4	Dynamic Programming (1D, 2D)	Medium-Hard
5	Advanced: KMP, Number Theory, Heaps, Geometry	Hard

Level 1: Foundations (Easy)

FlashCard 1.1: Array Index Formulas

Concept: These formulas help navigate array-based data structures like heaps and grids.

Q: How do you find parent, left child, and right child in a binary heap/tree (at index i)?

Node	Formula
Parent	$\text{floor}((i - 1) / 2)$
Left Child	$2 * i + 1$
Right Child	$2 * i + 2$

JavaScript:

```
const parent = (i) => Math.floor((i - 1) / 2);
const leftChild = (i) => 2 * i + 1;
const rightChild = (i) => 2 * i + 2;
```

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Kth Largest Element in a Stream	#703	Easy
Last Stone Weight	#1046	Easy

FlashCard 1.1 Practice: Kth Largest Element in a Stream

Problem: Design a class to find the kth largest element in a stream.

```
class KthLargest {
  constructor(k, nums) {
    this.k = k;
    this.heap = new MinHeap();
    for (const num of nums) {
      this.heap.insert(num);
      if (this.heap.heap.length > k) this.heap.extractMin();
    }
  }

  add(val) {
    this.heap.insert(val);
    if (this.heap.heap.length > this.k) this.heap.extractMin();
    return this.heap.heap[0];
  }
}
```

Key Insight: Use a min-heap of size k to track k largest elements.

FlashCard 1.2: 2D to 1D Array Mapping

Concept: Flattening a 2D matrix into a 1D array for storage or traversal.

Q: How do you convert 2D coordinates to 1D index?

Method	Formula
Row-major (most common)	$\text{index} = \text{row} * \text{cols} + \text{col}$
Column-major	$\text{index} = \text{col} * \text{rows} + \text{row}$

JavaScript:

```
const to1D = (row, col, cols) => row * cols + col;
const to2D = (index, cols) => ({
  row: Math.floor(index / cols),
  col: index % cols
});
```

Practice Problem:

Problem	LeetCode #	Difficulty
Reshape the Matrix	#566	Easy
Image Smoother	#661	Easy

FlashCard 1.2 Practice: Reshape the Matrix

Problem: Reshape a matrix from rows×cols to newRows×newCols.

```
const matrixReshape = (mat, r, c) => {
  const m = mat.length, n = mat[0].length;
  if (m * n !== r * c) return mat;

  const result = Array(r).fill(null).map(() => Array(c).fill(0));
  for (let i = 0; i < m * n; i++) {
    const oldRow = Math.floor(i / n), oldCol = i % n;
    const newRow = Math.floor(i / c), newCol = i % c;
    result[newRow][newCol] = mat[oldRow][oldCol];
  }
  return result;
};
```

Key Insight: Convert 2D indices to 1D, then back to 2D.

FlashCard 1.3: Sub-Grid Index (Sudoku Pattern)

Concept: Used for problems like Sudoku where you need to identify which k×k subgrid a cell belongs to.

Q: What's the formula to get the subgrid index for a cell at (row, col) in an N×N board divided into k×k subgrids?

```
subGridIndex = floor(row/k) * k + floor(col/k)
```

Derivation:

For 9×9 board with 3×3 boxes (k=3):

Step 1: $\text{floor}(\text{row}/3) \rightarrow$ which box ROW (0, 1, or 2)

$\text{floor}(0-2/3) = 0$ (top row of boxes)

$\text{floor}(3-5/3) = 1$ (middle row)

$\text{floor}(6-8/3) = 2$ (bottom row)

Step 2: $\text{floor}(\text{col}/3) \rightarrow$ which box COLUMN

Step 3: $\text{boxIndex} = \text{boxRow} * k + \text{boxCol}$

Visual for 9×9:

	col	0	1	2	3	4	5	6	7	8
row 0-2		0		1		2				
row 3-5		3		4		5				
row 6-8		6		7		8				

JavaScript:

```
// For 9×9 Sudoku (k=3)
```

```
const getBoxIndex = (row, col) => Math.floor(row / 3) * 3 + Math.floor(col / 3);
```

```
// Generalized
```

```
const getSubGridIndex = (row, col, k) => Math.floor(row / k) * k + Math.floor(col / k);
```

Why It Matters:

- Grid partitioning in DFS/graph problems
- Track constraints in Sudoku
- Extends to 3D: $\text{floor}(x/k)*k*k + \text{floor}(y/k)*k + z$

Practice Problem:

Problem	LeetCode #	Difficulty
Valid Sudoku	#36	Medium
Sudoku Solver	#37	Hard

FlashCard 1.3 Practice: Valid Sudoku

Problem: Determine if a 9×9 Sudoku board is valid.

```
const isValidSudoku = (board) => {
  const seen = new Set();

  for (let i = 0; i < 9; i++) {
    for (let j = 0; j < 9; j++) {
      const val = board[i][j];
      if (val === '.') continue;

      const boxIdx = Math.floor(i / 3) * 3 + Math.floor(j / 3);

      if (seen.has(`row${i}${val}`) ||
          seen.has(`col${j}${val}`) ||
          seen.has(`box${boxIdx}${val}`)) {
        return false;
      }
      seen.add(`row${i}${val}`);
      seen.add(`col${j}${val}`);
      seen.add(`box${boxIdx}${val}`);
    }
  }
  return true;
};
```

Key Insight: Use box index formula to track 3 constraints simultaneously.

FlashCard 1.4: Basic Math Formulas

Q: What are the essential basic math formulas?

Formula	Description
$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$	Euclidean algorithm for GCD
$\text{lcm}(a, b) = (a * b) / \text{gcd}(a, b)$	Least Common Multiple
$\text{sum} = n * (\text{first} + \text{last}) / 2$	Sum of Arithmetic Progression
$\text{sum} = \text{first} * (r^n - 1) / (r - 1)$	Sum of Geometric Progression

JavaScript:

```
// GCD - Euclidean
const gcd = (a, b) => b === 0 ? a : gcd(b, a % b);

// LCM
const lcm = (a, b) => Math.abs(a * b) / gcd(a, b);

// Sum of 1 to n
const sum1toN = (n) => (n * (n + 1)) / 2;

// Sum of AP
const sumAP = (n, first, last) => (n * (first + last)) / 2;

// Sum of GP
const sumGP = (first, r, n) => {
  if (r === 1) return first * n;
  return first * (Math.pow(r, n) - 1) / (r - 1);
};
```

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Greatest Common Divisor	#1911 (Bit)	Easy
LCM of Array	#	-

FlashCard 1.4 Practice: GCD in Real Problems

Problem: Find GCD of array elements.

```
const findGCD = (nums) => {  
  const min = Math.min(...nums);  
  const max = Math.max(...nums);  
  
  const gcd = (a, b) => b === 0 ? a : gcd(b, a % b);  
  return gcd(min, max);  
};
```

Key Insight: GCD of whole array = GCD(min, max). Used in reducing fractions, finding common factors.

FlashCard 1.5: Prime Number Check

Concept: Checking if a number is prime efficiently by testing divisibility up to $\sqrt{n}$.

Q: How do you check if n is prime efficiently?

```
function isPrime(n) {  
  if (n <= 1) return false;  
  if (n <= 3) return true;  
  if (n % 2 === 0 || n % 3 === 0) return false;  
  
  for (let i = 5; i * i <= n; i += 6) {  
    if (n % i === 0 || n % (i + 2) === 0) return false;  
  }  
  return true;  
}
```

Why $i += 6$? All primes > 3 are of form $6k \pm 1$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Count Primes	#204	Medium

Problem	LeetCode #	Difficulty
Ugly Number	#263	Easy

FlashCard 1.5 Practice: Count Primes

Problem: Count the number of prime numbers less than n.

```
const countPrimes = (n) => {
  if (n <= 2) return 0;
  const isPrime = new Array(n).fill(true);
  isPrime[0] = isPrime[1] = false;

  for (let i = 2; i * i < n; i++) {
    if (isPrime[i]) {
      for (let j = i * i; j < n; j += i) {
        isPrime[j] = false;
      }
    }
  }
  return isPrime.filter(Boolean).length;
};
```

Key Insight: Sieve of Eratosthenes - mark multiples of each prime as non-prime.

FlashCard 1.6: Basic String Operations

Q: How do you check if two strings are anagrams?

```
// Method 1: Sort – O(n log n)
const isAnagram = (s, t) =>
  s.split('').sort().join('') === t.split('').sort().join('');

// Method 2: Frequency Map – O(n)
const isAnagram = (s, t) => {
  if (s.length !== t.length) return false;
  const count = {};
  for (const c of s) count[c] = (count[c] || 0) + 1;
  for (const c of t) {
    if (!count[c]) return false;
    count[c]--;
  }
  return true;
};
```

Q: How do you reverse a number?

```
const reverse = (n) =>
  parseInt(n.toString().split('').reverse().join('')) * Math.sign(n);
```

Q: How do you check if a number is a palindrome?

```
const isPalindrome = (x) => {
  const str = x.toString();
  return str === str.split('').reverse().join('');
};
```

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Valid Anagram	#242	Easy
Group Anagrams	#49	Medium
Palindrome Number	#9	Easy

FlashCard 1.6 Practice: Group Anagrams

Problem: Group anagrams together from an array of strings.

```
const groupAnagrams = (strs) => {
  const map = new Map();

  for (const s of strs) {
    const key = s.split('').sort().join('');
    if (!map.has(key)) map.set(key, []);
    map.get(key).push(s);
  }

  return Array.from(map.values());
};
```

Key Insight: Anagrams sort to the same string - use as map key.

FlashCard 1.7: Basic Bit Operations

Concept: Essential bit manipulation tricks using AND, OR, XOR.

Q: What are the fundamental bit operations?

Operation	Code
Check if power of 2	n > 0 && (n & (n - 1)) === 0
Get bit at position i	(num >> i) & 1
Set bit at position i	num (1 << i)
Clear bit at position i	num & ~(1 << i)
Count set bits	Use n & (n - 1) (clears lowest bit)

```
// Power of 2
const isPowerOfTwo = (n) => n > 0 && (n & (n - 1)) === 0;

// Count set bits (Brian Kernighan's)
const hammingWeight = (n) => {
  let count = 0;
  while (n !== 0) {
    n = n & (n - 1);
    count++;
  }
  return count;
};

// Single number (XOR trick) - all pairs cancel out
const singleNumber = (nums) => nums.reduce((a, b) => a ^ b, 0);
```

XOR Properties: $a \oplus a = 0$, $a \oplus 0 = a$, $a \oplus b \oplus a = b$

Practice Problem:

Problem	LeetCode #	Difficulty
Single Number	#136	Easy
Single Number II	#137	Medium
Missing Number	#268	Easy

FlashCard 1.7 Practice: Single Number II

Problem: Find element that appears once while others appear 3 times.

```
const singleNumber = (nums) => {
  let ones = 0, twos = 0;
  for (const num of nums) {
    ones = (ones ^ num) & ~twos;
    twos = (twos ^ num) & ~ones;
  }
  return ones;
};
```

Key Insight: Use bit manipulation to track bits that appear once, twice, and three times.

FlashCard 1.8: Sum/Product of Array

Concept: Basic array operations that are foundation for many algorithms.

```
const sum = arr => arr.reduce((a, b) => a + b, 0);
```

```
const product = (arr, mod) =>  
  arr.reduce((a, b) => (a * b) % mod, 1);
```

Practice Problem:

Problem	LeetCode #	Difficulty
Running Sum of 1d Array	#1480	Easy
Product of Array Except Self	#238	Medium

FlashCard 1.8 Practice: Product of Array Except Self

Problem: Calculate product of all elements except self (no division).

```

const productExceptSelf = (nums) => {
  const n = nums.length;
  const result = Array(n).fill(1);

  let prefix = 1;
  for (let i = 0; i < n; i++) {
    result[i] = prefix;
    prefix *= nums[i];
  }

  let suffix = 1;
  for (let i = n - 1; i >= 0; i--) {
    result[i] *= suffix;
    suffix *= nums[i];
  }

  return result;
};

```

Key Insight: Use prefix and suffix products. Two passes - left to right, then right to left.

Level 2: Simple Algorithms (Easy-Medium)

FlashCard 2.1: Two Pointers - Remove Duplicates

Concept: Use slow/fast pointers to remove duplicates in-place in a sorted array.

Q: How do you remove duplicates from a sorted array in-place?

```
const removeDuplicates = (nums) => {
  if (nums.length === 0) return 0;
  let slow = 0;
  for (let fast = 1; fast < nums.length; fast++) {
    if (nums[fast] !== nums[slow]) {
      slow++;
      nums[slow] = nums[fast];
    }
  }
  return slow + 1;
};
```

Time: $O(n)$, **Space:** $O(1)$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Remove Duplicates from Sorted Array	#26	Easy
Remove Duplicates II	#80	Medium

FlashCard 2.1 Practice: Remove Duplicates II

Problem: Remove duplicates allowing at most 2 occurrences.

```
const removeDuplicates = (nums) => {
  if (nums.length <= 2) return nums.length;

  let slow = 1;
  for (let fast = 2; fast < nums.length; fast++) {
    if (nums[fast] !== nums[slow - 1]) {
      slow++;
      nums[slow] = nums[fast];
    }
  }
  return slow + 1;
};
```


Key Insight: Keep track of 2 previous elements to allow 2 duplicates.

FlashCard 2.2: Two Pointers - Container With Most Water

Concept: Two pointers starting from both ends, always move the shorter line.

Q: How do you find the container that holds the most water?

```
const maxArea = (height) => {  
  let left = 0, right = height.length - 1, maxArea = 0;  
  
  while (left < right) {  
    const width = right - left;  
    const h = Math.min(height[left], height[right]);  
    maxArea = Math.max(maxArea, width * h);  
  
    if (height[left] < height[right]) left++;  
    else right--;  
  }  
  return maxArea;  
};
```

Why move shorter? The width is decreasing, so we must increase height to potentially find a better area.

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Container With Most Water	#11	Medium
Trapping Rain Water	#42	Hard

FlashCard 2.2 Practice: Trapping Rain Water

Problem: Calculate how much water can be trapped between bars.

```

const trap = (height) => {
  let left = 0, right = height.length - 1;
  let leftMax = 0, rightMax = 0, water = 0;

  while (left < right) {
    if (height[left] < height[right]) {
      if (height[left] >= leftMax) leftMax = height[left];
      else water += leftMax - height[left];
      left++;
    } else {
      if (height[right] >= rightMax) rightMax = height[right];
      else water += rightMax - height[right];
      right--;
    }
  }
  return water;
};

```

Key Insight: Two pointers track max from each side. Water at each position = $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$.

FlashCard 2.3: Binary Search - Basic

Concept: Find a target in a sorted array by repeatedly dividing the search space in half.

Q: What are the formulas for lower bound and upper bound?

```
// Lower Bound: first element >= target
const lowerBound = (arr, target) => {
  let left = 0, right = arr.length;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] < target) left = mid + 1;
    else right = mid;
  }
  return left;
};
```

```
// Upper Bound: first element > target
const upperBound = (arr, target) => {
  let left = 0, right = arr.length;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] <= target) left = mid + 1;
    else right = mid;
  }
  return left;
};
```

Key Insight: Use `left + Math.floor((right - left) / 2)` to avoid overflow.

 Practice Problem:

Problem	LeetCode #	Difficulty
Search Insert Position	#35	Easy
Find First and Last Position	#34	Medium
Peak Index in Mountain Array	#852	Medium

FlashCard 2.3 Practice: Find First and Last Position

Problem: Find the start and end position of target in sorted array.

```

const searchRange = (nums, target) => {
  const findBound = (isFirst) => {
    let left = 0, right = nums.length;
    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (nums[mid] > target || (isFirst && nums[mid] >= target)) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  };

  const first = findBound(true);
  const last = findBound(false) - 1;

  if (first <= last && nums[first] === target) return [first, last];
  return [-1, -1];
};

```

Key Insight: Use lower/upper bound to find first and last positions.

FlashCard 2.4: Binary Search in Rotated Array

Concept: A rotated sorted array is split into two sorted halves. Determine which half is sorted.

Q: How do you search in a rotated sorted array?

```

const search = (nums, target) => {
  let left = 0, right = nums.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (nums[mid] === target) return mid;

    // Left half is sorted
    if (nums[left] <= nums[mid]) {
      if (target >= nums[left] && target < nums[mid]) {
        right = mid - 1;
      } else {
        left = mid + 1;
      }
    }
    // Right half is sorted
    else {
      if (target > nums[mid] && target <= nums[right]) {
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }
  }
  return -1;
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Search in Rotated Sorted Array	#33	Medium
Find Minimum in Rotated Array	#153	Medium

FlashCard 2.4 Practice: Find Minimum in Rotated Array

Problem: Find minimum element in rotated sorted array.

```

const findMin = (nums) => {
  let left = 0, right = nums.length - 1;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (nums[mid] > nums[right]) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }
  return nums[left];
};

```

Key Insight: If $mid > right$, minimum is in right half. Otherwise in left half.

FlashCard 2.5: Sliding Window - Fixed Size

Concept: Maintain a window of size k , slide it across the array.

Q: How do you find the maximum sum of a fixed-size window?

```

const maxSumSubarray = (arr, k) => {
  let sum = 0;
  for (let i = 0; i < k; i++) sum += arr[i];
  let maxSum = sum;

  for (let i = k; i < arr.length; i++) {
    sum += arr[i] - arr[i - k];
    maxSum = Math.max(maxSum, sum);
  }
  return maxSum;
};

```

Time: $O(n)$, **Space:** $O(1)$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Maximum Average Subarray	#643	Easy
Maximum Number of Vowels	#1456	Medium

FlashCard 2.5 Practice: Maximum Average Subarray

Problem: Find maximum average of any contiguous subarray of length k.

```
const findMaxAverage = (nums, k) => {  
  let sum = 0;  
  for (let i = 0; i < k; i++) sum += nums[i];  
  let maxSum = sum;  
  
  for (let i = k; i < nums.length; i++) {  
    sum += nums[i] - nums[i - k];  
    maxSum = Math.max(maxSum, sum);  
  }  
  return maxSum / k;  
};
```

Key Insight: Sliding window - subtract leftmost, add rightmost as you slide.

FlashCard 2.6: Sliding Window - Minimum Window Substring

Concept: Expand right pointer to satisfy condition, then shrink from left.

Q: How do you find the minimum window containing all characters of pattern in string?

```

const minWindow = (s, t) => {
  const need = {}, window = {};
  for (const c of t) need[c] = (need[c] || 0) + 1;

  let have = 0, needCount = Object.keys(need).length;
  let left = 0, result = [-1, -1], resultLen = Infinity;

  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    window[c] = (window[c] || 0) + 1;
    if (need[c] && window[c] === need[c]) have++;

    while (have === needCount) {
      if (right - left + 1 < resultLen) {
        result = [left, right];
        resultLen = right - left + 1;
      }
      window[s[left]]--;
      if (need[s[left]] && window[s[left]] < need[s[left]]) have--;
      left++;
    }
  }
  return resultLen === Infinity ? '' : s.slice(result[0], result[1] + 1);
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Minimum Window Substring	#76	Hard
Permutation in String	#567	Medium

FlashCard 2.6 Practice: Permutation in String

Problem: Check if s1's permutation is a substring of s2.


```

const checkInclusion = (s1, s2) => {
  const need = {}, window = {};
  for (const c of s1) need[c] = (need[c] || 0) + 1;

  let have = 0, needCount = Object.keys(need).length;
  let left = 0;

  for (let right = 0; right < s2.length; right++) {
    const c = s2[right];
    window[c] = (window[c] || 0) + 1;
    if (need[c] && window[c] === need[c]) have++;

    while (have === needCount) {
      if (right - left + 1 === s1.length) return true;
      window[s[left]]--;
      if (need[s[left]] && window[s[left]] < need[s[left]]) have--;
      left++;
    }
  }
  return false;
};

```

Key Insight: Fixed-size sliding window - check if all chars match.

FlashCard 2.7: Longest Substring Without Repeating

Concept: Sliding window with a Set to track unique characters.

Q: How do you find the length of longest substring without repeating characters?

```
const lengthOfLongestSubstring = (s) => {  
  const seen = new Set();  
  let maxLen = 0, left = 0;  
  
  for (let right = 0; right < s.length; right++) {  
    while (seen.has(s[right])) {  
      seen.delete(s[left++]);  
    }  
    seen.add(s[right]);  
    maxLen = Math.max(maxLen, right - left + 1);  
  }  
  return maxLen;  
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Longest Substring Without Repeating	#3	Medium
Longest Repeating Character	#424	Medium

FlashCard 2.7 Practice: Longest Repeating Character

Problem: Find longest repeating character after at most k replacements.

```

const characterReplacement = (s, k) => {
  const count = {};
  let maxCount = 0, left = 0, result = 0;

  for (let right = 0; right < s.length; right++) {
    count[s[right]] = (count[s[right]] || 0) + 1;
    maxCount = Math.max(maxCount, count[s[right]]);

    while (right - left + 1 - maxCount > k) {
      count[s[left]]--;
      left++;
    }
    result = Math.max(result, right - left + 1);
  }
  return result;
};

```

Key Insight: Maintain window where $(\text{windowSize} - \text{maxCount}) \leq k$. Track max frequency in window.

FlashCard 2.8: Merge Sort (Divide & Conquer)

Concept: Recursively divide array in half, then merge sorted halves.

Q: How does merge sort work?

```

const mergeSort = (arr) => {
  if (arr.length <= 1) return arr;

  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));

  return merge(left, right);
};

const merge = (left, right) => {
  const result = [];
  let i = 0, j = 0;

  while (i < left.length && j < right.length) {
    if (left[i] <= right[j]) result.push(left[i++]);
    else result.push(right[j++]);
  }
  return [...result, ...left.slice(i), ...right.slice(j)];
};

```

Time: $O(n \log n)$, **Space:** $O(n)$

Practice Problem:

Problem	LeetCode #	Difficulty
Merge Intervals	#56	Medium
Sort List	#148	Medium

FlashCard 2.8 Practice: Merge Intervals

Problem: Merge all overlapping intervals.

```

const merge = (intervals) => {
  if (intervals.length <= 1) return intervals;

  intervals.sort((a, b) => a[0] - b[0]);
  const result = [intervals[0]];

  for (let i = 1; i < intervals.length; i++) {
    const last = result[result.length - 1];
    if (intervals[i][0] <= last[1]) {
      last[1] = Math.max(last[1], intervals[i][1]);
    } else {
      result.push(intervals[i]);
    }
  }
  return result;
};

```

Key Insight: Sort by start time, then merge overlapping intervals.

FlashCard 2.9: Quick Sort

Concept: Pick a pivot, partition around it, recursively sort partitions.

Q: How does quick sort work?

```

const quickSort = (arr, low = 0, high = arr.length - 1) => {
  if (low < high) {
    const pi = partition(arr, low, high);
    quickSort(arr, low, pi - 1);
    quickSort(arr, pi + 1, high);
  }
  return arr;
};

const partition = (arr, low, high) => {
  const pivot = arr[high];
  let i = low - 1;

  for (let j = low; j < high; j++) {
    if (arr[j] <= pivot) {
      i++;
      [arr[i], arr[j]] = [arr[j], arr[i]];
    }
  }
  [arr[i + 1], arr[high]] = [arr[high], arr[i + 1]];
  return i + 1;
};

```

Time: $O(n \log n)$ avg, $O(n^2)$ worst, **Space:** $O(\log n)$

Practice Problem:

Problem	LeetCode #	Difficulty
Kth Largest Element	#215	Medium
Top K Frequent Elements	#347	Medium

FlashCard 2.9 Practice: Kth Largest Element

Problem: Find the kth largest element in an array.

```

const findKthLargest = (nums, k) => {
  const quickSelect = (left, right, k) => {
    const pivot = nums[right];
    let p = left;

    for (let i = left; i < right; i++) {
      if (nums[i] >= pivot) {
        [nums[i], nums[p]] = [nums[p], nums[i]];
        p++;
      }
    }
    [nums[p], nums[right]] = [nums[right], nums[p]];

    if (p === k) return nums[p];
    if (p < k) return quickSelect(p + 1, right, k);
    return quickSelect(left, p - 1, k);
  };

  return quickSelect(0, nums.length - 1, k - 1);
};

```

Key Insight: Partition like quick sort, but only recurse on one side.

FlashCard 2.10: Linked List - Reverse

Concept: Iteratively or recursively reverse the next pointers.

Q: How do you reverse a linked list?

```
// Iterative
const reverseList = (head) => {
  let prev = null, curr = head;
  while (curr) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
};

// Recursive
const reverseList = (head) => {
  if (!head || !head.next) return head;
  const newHead = reverseList(head.next);
  head.next.next = head;
  head.next = null;
  return newHead;
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Reverse Linked List	#206	Easy
Reverse Linked List II	#92	Medium

FlashCard 2.10 Practice: Reverse Linked List II

Problem: Reverse a linked list from position left to right.


```

const reverseBetween = (head, left, right) => {
  const dummy = new ListNode(0);
  dummy.next = head;
  let prev = dummy;

  for (let i = 1; i < left; i++) prev = prev.next;

  let curr = prev.next;
  for (let i = 0; i < right - left; i++) {
    const next = curr.next;
    curr.next = next.next;
    next.next = prev.next;
    prev.next = next;
  }

  return dummy.next;
};

```

Key Insight: Track three pointers: prev (before reversal), curr (start), and next.

FlashCard 2.11: Detect Cycle in Linked List (Floyd's Algorithm)

Concept: Two pointers - slow moves 1 step, fast moves 2 steps. If they meet, there's a cycle.

Q: How do you detect a cycle in a linked list?

```

const hasCycle = (head) => {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
};

```

Q: How do you find the cycle start node?

```

const detectCycle = (head) => {
  let slow = head, fast = head;

  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) break;
  }
  if (!fast || !fast.next) return null;

  // Move both 1 step to find cycle start
  slow = head;
  while (slow !== fast) {
    slow = slow.next;
    fast = fast.next;
  }
  return slow;
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Linked List Cycle	#141	Easy
Find the Duplicate Number	#287	Medium

FlashCard 2.11 Practice: Find the Duplicate Number

Problem: Find duplicate number in array using Floyd's algorithm.

```
const findDuplicate = (nums) => {  
  let slow = nums[0], fast = nums[0];  
  
  do {  
    slow = nums[slow];  
    fast = nums[nums[fast]];  
  } while (slow !== fast);  
  
  slow = nums[0];  
  while (slow !== fast) {  
    slow = nums[slow];  
    fast = nums[fast];  
  }  
  return slow;  
};
```

Key Insight: Treat array as linked list where `nums[i]` points to next node.

Level 3: Data Structures (Medium)

FlashCard 3.1: Tree - Node Depth & Height

Concept: Depth = edges from root to node. Height = edges from node to deepest leaf.

Q: How do you calculate depth and height of a tree node?

```
// Height (recursive)
const getHeight = (node) => {
  if (!node) return -1; // or 0 for node count
  return 1 + Math.max(getHeight(node.left), getHeight(node.right));
};

// Depth - BFS from root
const getDepth = (root, target) => {
  if (!root) return -1;
  const queue = [[root, 0]];
  while (queue.length) {
    const [node, depth] = queue.shift();
    if (node === target) return depth;
    if (node.left) queue.push([node.left, depth + 1]);
    if (node.right) queue.push([node.right, depth + 1]);
  }
  return -1;
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Maximum Depth of Binary Tree	#104	Easy
Minimum Depth of Binary Tree	#111	Easy

FlashCard 3.1 Practice: Balanced Binary Tree

Problem: Determine if binary tree is height-balanced.

```

const isBalanced = (root) => {
  const dfs = (node) => {
    if (!node) return 0;
    const left = dfs(node.left);
    if (left === -1) return -1;
    const right = dfs(node.right);
    if (right === -1) return -1;
    return Math.abs(left - right) > 1 ? -1 : 1 + Math.max(left, right);
  };
  return dfs(root) !== -1;
};

```

Key Insight: Return height or -1 if unbalanced. Check balance at each node.

FlashCard 3.2: Tree - Node Count

Q: How do you count nodes in a binary tree?

```

// BFS
const countNodesBFS = (root) => {
  if (!root) return 0;
  let count = 0;
  const queue = [root];
  while (queue.length) {
    const node = queue.shift();
    count++;
    if (node.left) queue.push(node.left);
    if (node.right) queue.push(node.right);
  }
  return count;
};

```

```

// DFS
const countNodesDFS = (root) => {
  if (!root) return 0;
  return 1 + countNodesDFS(root.left) + countNodesDFS(root.right);
};

```

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Count Complete Tree Nodes	#222	Medium
Invert Binary Tree	#226	Easy

FlashCard 3.2 Practice: Count Complete Tree Nodes

Problem: Count nodes in a complete binary tree.

```
const countNodes = (root) => {
  if (!root) return 0;

  const getDepth = (node) => {
    let depth = 0;
    while (node) {
      depth++;
      node = node.left;
    }
    return depth;
  };

  const leftDepth = getDepth(root.left);
  const rightDepth = getDepth(root.right);

  if (leftDepth === rightDepth) {
    return (1 << leftDepth) + countNodes(root.right);
  } else {
    return (1 << rightDepth) + countNodes(root.left);
  }
};
```

Key Insight: Complete tree has $2^h - 1$ nodes if $\text{leftDepth} === \text{rightDepth}$.

FlashCard 3.3: Valid Parentheses

Concept: Use a stack to match opening and closing brackets.

Q: How do you validate balanced parentheses?

```
const isValid = (s) => {  
  const stack = [];  
  const map = { ')': '(', '}': '{', ']': '[' };  
  
  for (const c of s) {  
    if (c in map) {  
      if (stack.pop() !== map[c]) return false;  
    } else {  
      stack.push(c);  
    }  
  }  
  return stack.length === 0;  
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Valid Parentheses	#20	Easy
Generate Parentheses	#22	Medium

FlashCard 3.3 Practice: Generate Parentheses

Problem: Generate all combinations of well-formed parentheses.

```

const generateParenthesis = (n) => {
  const result = [];

  const backtrack = (open, close, current) => {
    if (open === n && close === n) {
      result.push(current);
      return;
    }
    if (open < n) backtrack(open + 1, close, current + '(');
    if (close < open) backtrack(open, close + 1, current + ')');
  };

  backtrack(0, 0, '');
  return result;
};

```

Key Insight: At each step, add '(' if open < n, add ')' if close < open.

FlashCard 3.4: Largest Rectangle in Histogram

Concept: Use monotonic increasing stack to find largest rectangle.

Q: How do you find the largest rectangle in a histogram?

```

const largestRectangleArea = (heights) => {
  const stack = [];
  let maxArea = 0;

  for (let i = 0; i <= heights.length; i++) {
    const h = i === heights.length ? 0 : heights[i];
    while (stack.length && heights[stack[stack.length - 1]] > h) {
      const height = heights[stack.pop()];
      const width = stack.length ? i - stack[stack.length - 1] - 1 : i;
      maxArea = Math.max(maxArea, height * width);
    }
    stack.push(i);
  }
  return maxArea;
};

```


Time: $O(n)$, **Space:** $O(n)$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Largest Rectangle in Histogram	#84	Hard
Maximal Rectangle	#85	Hard

FlashCard 3.4 Practice: Maximal Rectangle

Problem: Find largest rectangle of 1s in a binary matrix.

```

const maximalRectangle = (matrix) => {
  if (!matrix.length) return 0;

  const heights = new Array(matrix[0].length).fill(0);
  let maxArea = 0;

  for (const row of matrix) {
    for (let j = 0; j < row.length; j++) {
      heights[j] = row[j] === '1' ? heights[j] + 1 : 0;
    }
    maxArea = Math.max(maxArea, largestRectangleArea(heights));
  }
  return maxArea;
};

const largestRectangleArea = (heights) => {
  const stack = [];
  let maxArea = 0;
  for (let i = 0; i <= heights.length; i++) {
    const h = i === heights.length ? 0 : heights[i];
    while (stack.length && heights[stack[stack.length - 1]] > h) {
      const height = heights[stack.pop()];
      const width = stack.length ? i - stack[stack.length - 1] - 1 : i;
      maxArea = Math.max(maxArea, height * width);
    }
    stack.push(i);
  }
  return maxArea;
};

```

Key Insight: Treat each row as histogram base, apply largest rectangle.

FlashCard 3.5: Number of Islands (DFS/BFS)

Concept: Mark visited cells to avoid counting the same island twice.

Q: How do you count the number of islands in a grid?

```
// DFS
const numIslands = (grid) => {
  let count = 0;

  const dfs = (r, c) => {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length) return;
    if (grid[r][c] === '0') return;

    grid[r][c] = '0'; // Mark visited
    dfs(r + 1, c);
    dfs(r - 1, c);
    dfs(r, c + 1);
    dfs(r, c - 1);
  };

  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[0].length; c++) {
      if (grid[r][c] === '1') {
        count++;
        dfs(r, c);
      }
    }
  }
  return count;
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Number of Islands	#200	Medium
Max Area of Island	#695	Medium
Flood Fill	#733	Easy

FlashCard 3.5 Practice: Max Area of Island

Problem: Find maximum area of an island in grid.

```

const maxAreaOfIsland = (grid) => {
  let maxArea = 0;

  const dfs = (r, c) => {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length) return 0;
    if (grid[r][c] === 0) return 0;

    grid[r][c] = 0;
    return 1 + dfs(r + 1, c) + dfs(r - 1, c) + dfs(r, c + 1) + dfs(r, c - 1);
  };

  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[0].length; c++) {
      if (grid[r][c] === 1) {
        maxArea = Math.max(maxArea, dfs(r, c));
      }
    }
  }
  return maxArea;
};

```

Key Insight: DFS explores entire island, returns area. Mark visited by setting to 0.

FlashCard 3.6: BFS Shortest Path

Concept: Use a queue, track visited nodes and distance.

Q: How do you find shortest path in an unweighted graph?

```

const shortestPath = (grid, start, target) => {
  const queue = [[start, 0]];
  const visited = new Set([start]);

  while (queue.length) {
    const [pos, dist] = queue.shift();
    if (pos === target) return dist;

    for (const neighbor of getNeighbors(pos)) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push([neighbor, dist + 1]);
      }
    }
  }
  return -1;
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Rotting Oranges	#994	Medium
Walls and Gates	#286	Medium

FlashCard 3.6 Practice: Rotting Oranges

Problem: Find minimum minutes for all oranges to rot.

```

const orangesRotting = (grid) => {
  const queue = [];
  let fresh = 0, minutes = 0;

  for (let r = 0; r < grid.length; r++) {
    for (let c = 0; c < grid[0].length; c++) {
      if (grid[r][c] === 2) queue.push([r, c, 0]);
      if (grid[r][c] === 1) fresh++;
    }
  }

  const dirs = [[0,1], [0,-1], [1,0], [-1,0]];
  while (queue.length) {
    const [r, c, time] = queue.shift();
    minutes = time;
    for (const [dr, dc] of dirs) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < grid.length && nc >= 0 && nc < grid[0].length && grid[nr][nc] === 1) {
        grid[nr][nc] = 2;
        fresh--;
        queue.push([nr, nc, time + 1]);
      }
    }
  }

  return fresh === 0 ? minutes : -1;
};

```

Key Insight: BFS from all rotten oranges simultaneously - multi-source BFS.

FlashCard 3.7: Union-Find (DSU)

Concept: Data structure for tracking disjoint sets. Used in Kruskal's MST and cycle detection.

Q: How do you implement Union-Find with path compression and union by rank?

```

class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);
    this.rank = new Array(n).fill(0);
  }

  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]); // Path compression
    }
    return this.parent[x];
  }

  union(x, y) {
    const rootX = this.find(x);
    const rootY = this.find(y);

    if (rootX === rootY) return false; // Cycle detected

    // Union by rank
    if (this.rank[rootX] < this.rank[rootY]) {
      this.parent[rootX] = rootY;
    } else if (this.rank[rootX] > this.rank[rootY]) {
      this.parent[rootY] = rootX;
    } else {
      this.parent[rootY] = rootX;
      this.rank[rootX]++;
    }
    return true;
  }
}

```

🎯 Practice Problem:

Problem	LeetCode #	Difficulty
Number of Connected Components	#323 (Premium)	Medium
Redundant Connection	#684	Medium

FlashCard 3.7 Practice: Redundant Connection

Problem: Find edge that creates a cycle in the graph.

```
const findRedundantConnection = (edges) => {
  const parent = Array(edges.length + 1).fill(0).map((_, i) => i);

  const find = (x) => {
    if (parent[x] !== x) parent[x] = find(parent[x]);
    return parent[x];
  };

  const union = (x, y) => {
    const rootX = find(x), rootY = find(y);
    if (rootX === rootY) return false;
    parent[rootX] = rootY;
    return true;
  };

  for (const [u, v] of edges) {
    if (!union(u, v)) return [u, v];
  }
};
```

Key Insight: If union fails (already connected), that edge creates the cycle.

Level 4: Dynamic Programming (Medium-Hard)

FlashCard 4.1: Fibonacci (Multiple Approaches)

Q: What are the different ways to compute Fibonacci?


```

// Recursive (exponential - BAD)
const fib = (n) => n <= 1 ? n : fib(n - 1) + fib(n - 2);

// Memoization (top-down) - O(n) time, O(n) space
const fibMemo = (n, memo = {}) => {
  if (n in memo) return memo[n];
  if (n <= 1) return n;
  return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
};

// Tabulation (bottom-up) - O(n) time, O(n) space
const fibTab = (n) => {
  if (n <= 1) return n;
  const dp = [0, 1];
  for (let i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
  }
  return dp[n];
};

// Space optimized - O(n) time, O(1) space
const fib = (n) => {
  if (n <= 1) return n;
  let prev2 = 0, prev1 = 1;
  for (let i = 2; i <= n; i++) {
    [prev2, prev1] = [prev1, prev2 + prev1];
  }
  return prev1;
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Climbing Stairs	#70	Easy
Min Cost Climbing Stairs	#746	Easy

FlashCard 4.1 Practice: Min Cost Climbing Stairs

Problem: Find minimum cost to reach top of stairs.

```
const minCostClimbingStairs = (cost) => {  
  const n = cost.length;  
  let prev2 = cost[0], prev1 = cost[1];  
  
  for (let i = 2; i < n; i++) {  
    const curr = cost[i] + Math.min(prev2, prev1);  
    prev2 = prev1;  
    prev1 = curr;  
  }  
  return Math.min(prev2, prev1);  
};
```

Key Insight: Same as Fibonacci but with costs. At each step, choose cheaper of 1 or 2 jumps.

FlashCard 4.2: Climbing Stairs

Concept: Same recurrence as Fibonacci. Can take 1 or 2 steps.

Q: How many ways to climb n stairs?

```
const climbStairs = (n) => {  
  if (n <= 2) return n;  
  let prev2 = 1, prev1 = 2;  
  for (let i = 3; i <= n; i++) {  
    [prev2, prev1] = [prev1, prev2 + prev1];  
  }  
  return prev1;  
};
```

Formula: $dp[i] = dp[i-1] + dp[i-2]$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
House Robber II	#213	Medium
House Robber III	#337	Medium

FlashCard 4.2 Practice: House Robber II

Problem: Houses in a circle, can't rob adjacent houses.

```
const rob = (nums) => {
  if (nums.length === 1) return nums[0];

  const robRange = (start, end) => {
    let prev2 = 0, prev1 = 0;
    for (let i = start; i <= end; i++) {
      const curr = Math.max(prev1, prev2 + nums[i]);
      prev2 = prev1;
      prev1 = curr;
    }
    return prev1;
  };

  return Math.max(robRange(0, nums.length - 2), robRange(1, nums.length - 1));
};
```

Key Insight: Exclude either first or last house, solve two subproblems.

FlashCard 4.3: House Robber

Concept: Don't rob two adjacent houses. At each house, choose max of (skip current, rob current + prev2).

Q: What's the maximum amount you can rob without robbing adjacent houses?

```
const rob = (nums) => {
  if (nums.length === 0) return 0;
  if (nums.length === 1) return nums[0];

  let prev2 = nums[0], prev1 = Math.max(nums[0], nums[1]);
  for (let i = 2; i < nums.length; i++) {
    [prev2, prev1] = [prev1, Math.max(prev1, prev2 + nums[i])];
  }
  return prev1;
};
```

Formula: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$

Practice Problem:

Problem	LeetCode #	Difficulty
Delete and Earn	#740	Medium
Coin Change	#322	Medium

FlashCard 4.3 Practice: Coin Change

Problem: Find fewest coins to make up amount.

```
const coinChange = (coins, amount) => {
  const dp = Array(amount + 1).fill(Infinity);
  dp[0] = 0;

  for (let i = 1; i <= amount; i++) {
    for (const coin of coins) {
      if (coin <= i) {
        dp[i] = Math.min(dp[i], dp[i - coin] + 1);
      }
    }
  }
  return dp[amount] === Infinity ? -1 : dp[amount];
};
```

Key Insight: Unbounded knapsack - each coin can be used unlimited times.

FlashCard 4.4: Longest Common Subsequence (LCS)

Concept: Compare two strings character by character. If match, add 1 to diagonal. Otherwise, take max from left or top.

Q: What's the formula for LCS?

```
dp[i][j] = dp[i-1][j-1] + 1  if chars match
dp[i][j] = max(dp[i-1][j], dp[i][j-1])  otherwise
```

```
const longestCommonSubsequence = (text1, text2) => {
  const m = text1.length, n = text2.length;
  const dp = Array(m + 1).fill(null).map(() => Array(n + 1).fill(0));

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (text1[i - 1] === text2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }
  return dp[m][n];
};
```

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Longest Common Subsequence	#1143	Medium
Shortest Common Supersequence	#1092	Hard

FlashCard 4.4 Practice: Shortest Common Supersequence

Problem: Find shortest common supersequence of two strings.

```
const shortestCommonSupersequence = (str1, str2) => {
  const m = str1.length, n = str2.length;
  const dp = Array(m + 1).fill(null).map(() => Array(n + 1).fill(0));

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (str1[i-1] === str2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
      else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
    }
  }

  let i = m, j = n, result = '';
  while (i > 0 && j > 0) {
    if (str1[i-1] === str2[j-1]) {
      result = str1[i-1] + result;
      i--; j--;
    } else if (dp[i-1][j] > dp[i][j-1]) {
      result = str1[i-1] + result;
      i--;
    } else {
      result = str2[j-1] + result;
      j--;
    }
  }
  return str1.slice(0, i) + str2.slice(0, j) + result;
};
```

Key Insight: Build LCS first, then add remaining characters.

FlashCard 4.5: Edit Distance (Levenshtein)

Concept: Transform one string to another using insert, delete, replace.

Q: What are the operations for edit distance?

Operation	Cost
Insert	$dp[i][j] = dp[i-1][j] + 1$
Delete	$dp[i][j] = dp[i][j-1] + 1$
Replace	$dp[i][j] = dp[i-1][j-1] + 1$ (if different)

```
const minDistance = (word1, word2) => {
  const m = word1.length, n = word2.length;
  const dp = Array(m + 1).fill(null).map(() => Array(n + 1).fill(0));

  for (let i = 0; i <= m; i++) dp[i][0] = i;
  for (let j = 0; j <= n; j++) dp[0][j] = j;

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (word1[i - 1] === word2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1];
      } else {
        dp[i][j] = 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
      }
    }
  }
  return dp[m][n];
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Edit Distance	#72	Hard
One Edit Distance	#161	Medium

FlashCard 4.5 Practice: One Edit Distance

Problem: Check if two strings are exactly one edit apart.

```

const isOneEditDistance = (s, t) => {
  const m = s.length, n = t.length;
  if (m > n) return isOneEditDistance(t, s);

  for (let i = 0; i < m; i++) {
    if (s[i] !== t[i]) {
      return s.slice(i + 1) === t.slice(i + 1) ||
        s.slice(i) === t.slice(i + 1) ||
        s.slice(i + 1) === t.slice(i + 1);
    }
  }
  return Math.abs(m - n) === 1;
};

```

Key Insight: Handle three cases: replace, insert, or delete.

Level 5: Advanced Topics (Hard)

FlashCard 5.1: KMP String Matching

Concept: Build LPS (Longest Prefix Suffix) array to skip comparisons after a mismatch.

Q: How does KMP work?

Time: $O(n + m)$, Space: $O(m)$


```

const strStr = (haystack, needle) => {
  if (!needle) return 0;

  const buildLPS = (pattern) => {
    const lps = Array(pattern.length).fill(0);
    let len = 0, i = 1;
    while (i < pattern.length) {
      if (pattern[i] === pattern[len]) {
        lps[i++] = ++len;
      } else if (len !== 0) {
        len = lps[len - 1];
      } else {
        lps[i++] = 0;
      }
    }
    return lps;
  };

  const lps = buildLPS(needle);
  let i = 0, j = 0;

  while (i < haystack.length) {
    if (haystack[i] === needle[j]) {
      i++; j++;
      if (j === needle.length) return i - j;
    } else if (j !== 0) {
      j = lps[j - 1];
    } else {
      i++;
    }
  }
  return -1;
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Implement strStr	#28	Easy
Repeated String Match	#686	Medium

FlashCard 5.1 Practice: Repeated String Match

Problem: Find minimum number of repetitions of A to make B a substring.

```
const repeatedStringMatch = (a, b) => {  
  const repeat = Math.ceil(b.length / a.length);  
  const aRepeated = a.repeat(repeat);  
  if (aRepeated.includes(b)) return repeat;  
  
  const aExtended = a.repeat(repeat + 1);  
  if (aExtended.includes(b)) return repeat + 1;  
  
  return -1;  
};
```

Key Insight: B must appear in A repeated 1 or 2 times max.

FlashCard 5.2: Rabin-Karp (Rolling Hash)

Concept: Use rolling hash to compare substrings. If hash matches, verify with direct comparison.

Q: How does Rabin-Karp work?

```

const rabinKarp = (text, pattern) => {
  const d = 256, q = 101; // prime numbers
  const n = text.length, m = pattern.length;
  let p = 0, t = 0, h = 1;

  for (let i = 0; i < m - 1; i++) h = (h * d) % q;

  for (let i = 0; i < m; i++) {
    p = (d * p + pattern.charCodeAt(i)) % q;
    t = (d * t + text.charCodeAt(i)) % q;
  }

  for (let i = 0; i <= n - m; i++) {
    if (p === t) {
      let match = true;
      for (let j = 0; j < m; j++) {
        if (text[i + j] !== pattern[j]) { match = false; break; }
      }
      if (match) return i;
    }
    if (i < n - m) {
      t = (d * (t - text.charCodeAt(i) * h) + text.charCodeAt(i + m)) % q;
      if (t < 0) t += q;
    }
  }
  return -1;
};

```

Practice Problem:

Problem	LeetCode #	Difficulty
Substring with Concatenation	#30	Hard
Minimum Window Substring	#76	Hard

FlashCard 5.2 Practice: Minimum Window Substring

Problem: Find minimum window containing all chars of pattern.

```

const minWindow = (s, t) => {
  const need = {};
  for (const c of t) need[c] = (need[c] || 0) + 1;

  let have = 0, needCount = Object.keys(need).length;
  let left = 0, result = [-1, -1], resultLen = Infinity;

  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    if (need[c]) {
      if ((need[c] || 0) === have) have++;
    }

    while (have === needCount) {
      if (right - left + 1 < resultLen) {
        result = [left, right];
        resultLen = right - left + 1;
      }
      if (need[s[left]] === have) have--;
      left++;
    }
  }
  return resultLen === Infinity ? '' : s.slice(result[0], result[1] + 1);
};

```

Key Insight: Expand right, shrink left when valid, track minimum.

FlashCard 5.3: Sieve of Eratosthenes

Concept: Find all primes up to n by marking multiples of each prime.

Q: How do you find all primes up to n?

```

const sieve = (n) => {
  const isPrime = new Array(n + 1).fill(true);
  isPrime[0] = isPrime[1] = false;

  for (let i = 2; i * i <= n; i++) {
    if (isPrime[i]) {
      for (let j = i * i; j <= n; j += i) {
        isPrime[j] = false;
      }
    }
  }
  return isPrime.map((p, i) => p ? i : -1).filter(i => i >= 0);
};

```

Time: $O(n \log \log n)$

Practice Problem:

Problem	LeetCode #	Difficulty
Ugly Number II	#264	Medium
Super Ugly Number	#313	Medium

FlashCard 5.3 Practice: Ugly Number II

Problem: Find nth ugly number (divisible by 2, 3, or 5 only).

```

const nthUglyNumber = (n) => {
  const ugly = [1];
  let i2 = 0, i3 = 0, i5 = 0;

  while (ugly.length < n) {
    const next2 = ugly[i2] * 2;
    const next3 = ugly[i3] * 3;
    const next5 = ugly[i5] * 5;
    const next = Math.min(next2, next3, next5);

    ugly.push(next);
    if (next === next2) i2++;
    if (next === next3) i3++;
    if (next === next5) i5++;
  }
  return ugly[n - 1];
};

```

Key Insight: Merge sorted ugly numbers from 3 pointers (x2, x3, x5).

FlashCard 5.4: Find All Divisors

Q: How do you find all divisors of a number?

```

const getDivisors = (n) => {
  const divisors = [];
  for (let i = 1; i * i <= n; i++) {
    if (n % i === 0) {
      divisors.push(i);
      if (i !== n / i) divisors.push(n / i);
    }
  }
  return divisors.sort((a, b) => a - b);
};

```

Time: $O(\sqrt{n})$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Find All Divisors	#	-
Self Dividing Numbers	#728	Easy

FlashCard 5.4 Practice: Perfect Number

Problem: Find all perfect numbers up to limit.

```
const checkPerfectNumber = (num) => {
  if (num <= 1) return false;
  let sum = 1;

  for (let i = 2; i * i <= num; i++) {
    if (num % i === 0) {
      sum += i;
      if (i !== num / i) sum += num / i;
    }
  }
  return sum === num;
};
```

Key Insight: Sum divisors up to $\sqrt{n}$, add both i and num/i .

FlashCard 5.5: Min/Max Heap Implementation

Concept: Complete binary tree where parent is always smaller (min-heap) or larger (max-heap) than children.

Q: How do you implement a Min Heap?

```

class MinHeap {
  constructor() { this.heap = []; }

  parent(i) { return Math.floor((i - 1) / 2); }
  left(i) { return 2 * i + 1; }
  right(i) { return 2 * i + 2; }

  insert(val) {
    this.heap.push(val);
    this.bubbleUp(this.heap.length - 1);
  }

  bubbleUp(i) {
    while (i > 0 && this.heap[this.parent(i)] > this.heap[i]) {
      [this.heap[i], this.heap[this.parent(i)]] =
        [this.heap[this.parent(i)], this.heap[i]];
      i = this.parent(i);
    }
  }

  extractMin() {
    if (this.heap.length === 0) return null;
    if (this.heap.length === 1) return this.heap.pop();

    const min = this.heap[0];
    this.heap[0] = this.heap.pop();
    this.bubbleDown(0);
    return min;
  }

  bubbleDown(i) {
    while (this.left(i) < this.heap.length) {
      let smallest = this.left(i);
      if (this.right(i) < this.heap.length &&
        this.heap[this.right(i)] < this.heap[smallest]) {
        smallest = this.right(i);
      }
      if (this.heap[i] <= this.heap[smallest]) break;
      [this.heap[i], this.heap[smallest]] = [this.heap[smallest], this.heap[i]];
      i = smallest;
    }
  }
}

```



```
}  
}
```

Practice Problem:

Problem	LeetCode #	Difficulty
Kth Largest Element	#215	Medium
Top K Frequent Words	#692	Medium

FlashCard 5.5 Practice: Top K Frequent Elements

Problem: Find k most frequent elements.

```
const topKFrequent = (nums, k) => {  
  const freq = {};  
  for (const num of nums) freq[num] = (freq[num] || 0) + 1;  
  
  const bucket = Array(nums.length + 1);  
  for (const [num, count] of Object.entries(freq)) {  
    if (!bucket[count]) bucket[count] = [];  
    bucket[count].push(num);  
  }  
  
  const result = [];  
  for (let i = bucket.length - 1; i >= 0 && result.length < k; i--) {  
    if (bucket[i]) result.push(...bucket[i]);  
  }  
  return result.slice(0, k);  
};
```

Key Insight: Bucket sort by frequency, then collect from highest.

FlashCard 5.6: Power with Modulo

Concept: Modular exponentiation to handle large numbers.

Q: How do you compute $(\text{base}^{\text{exp}}) \% \text{mod}$ efficiently?

```
const modPow = (base, exp, mod) => {  
  let result = 1;  
  base = base % mod;  
  while (exp > 0) {  
    if (exp % 2 === 1) result = (result * base) % mod;  
    exp = Math.floor(exp / 2);  
    base = (base * base) % mod;  
  }  
  return result;  
};
```

Time: $O(\log \text{exp})$

 **Practice Problem:**

Problem	LeetCode #	Difficulty
Super Power	#372	Medium
Modular Exponentiation	#	-

FlashCard 5.6 Practice: Super Pow

Problem: Calculate $a^b \bmod 1337$.

```

const superPow = (a, b) => {
  const MOD = 1337;

  const modPow = (base, exp) => {
    let result = 1;
    base = base % MOD;
    while (exp > 0) {
      if (exp % 2 === 1) result = (result * base) % MOD;
      exp = Math.floor(exp / 2);
      base = (base * base) % MOD;
    }
    return result;
  };

  let result = 1;
  for (const digit of b) {
    result = (modPow(result, 10) * modPow(a, digit)) % MOD;
  }
  return result;
};

```

Key Insight: Apply modular exponentiation digit by digit.

FlashCard 5.7: Geometry - Distance Formulas

Q: What are the distance formulas?

```
// Euclidean distance
const euclideanDistance = (p1, p2) =>
  Math.sqrt(Math.pow(p2.x - p1.x, 2) + Math.pow(p2.y - p1.y, 2));

// Manhattan distance
const manhattanDistance = (p1, p2) =>
  Math.abs(p2.x - p1.x) + Math.abs(p2.y - p1.y);

// Point in circle
const isInCircle = (px, py, cx, cy, r) => {
  return Math.pow(px - cx, 2) + Math.pow(py - cy, 2) <= r * r;
};

// Point in rectangle (axis-aligned)
const isInRectangle = (px, py, rect) => {
  const { x, y, width, height } = rect;
  return px >= x && px <= x + width && py >= y && py <= y + height;
};
```

Practice Problem:

Problem	LeetCode #	Difficulty
Valid Perfect Square	#367	Easy
Rectangle Overlap	#836	Easy

FlashCard 5.7 Practice: Valid Perfect Square

Problem: Check if a number is a perfect square.

```
const isPerfectSquare = (num) => {
  let left = 1, right = num;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (mid * mid === num) return true;
    if (mid * mid < num) left = mid + 1;
    else right = mid - 1;
  }
  return false;
};
```

Key Insight: Binary search for perfect square root.

Quick Reference Card

Problem Type	Key Formula/Pattern
Binary Search	<code>mid = left + floor((right - left) / 2)</code>
Two Pointers	<code>left=0, right=n-1; move shorter</code>
Sliding Window	expand right, contract when valid
BFS	<code>queue.push</code> , mark visited
DFS	recurse, backtrack
DP	<code>dp[i]</code> depends on <code>dp[i-1]</code> , <code>dp[i-2]</code>
Union-Find	find, union, path compression
Bit Ops	<code>n & (n-1)</code> clears lowest set bit
XOR	<code>a ^ a = 0</code> , <code>a ^ 0 = a</code>
Fibonacci	<code>dp[n] = dp[n-1] + dp[n-2]</code>
Box Index	<code>floor(r/3) * 3 + floor(c/3)</code>

Practice Problems by Difficulty

Easy (Level 1-2)

Problem	LeetCode #	FlashCard
Remove Duplicates	#26	2.1
Valid Parentheses	#20	3.3
Reverse Linked List	#206	2.10
Maximum Depth of BT	#104	3.1
Climbing Stairs	#70	4.2
Single Number	#136	1.7
Running Sum	#1480	1.8

Medium (Level 2-4)

Problem	LeetCode #	FlashCard
Container With Most Water	#11	2.2
3Sum	#15	2.2
Valid Sudoku	#36	1.3
Longest Substring	#3	2.7
Search in Rotated Array	#33	2.4
Merge Intervals	#56	2.8
Number of Islands	#200	3.5
House Robber	#198	4.3
Coin Change	#322	4.3
Kth Largest	#215	2.9
Top K Frequent	#347	5.5

Problem	LeetCode #	FlashCard
Edit Distance	#72	4.5

Hard (Level 5)

Problem	LeetCode #	FlashCard
Sudoku Solver	#37	1.3
Trapping Rain Water	#42	2.2
Minimum Window Substring	#76	2.6
Largest Rectangle	#84	3.4
Maximal Rectangle	#85	3.4
Edit Distance	#72	4.5

Pro Tip: For FAANG interviews, it's not just about knowing these formulas - understand WHY they work and be ready to derive them on the whiteboard!

Learning Path: Start with Level 1, master each FlashCard with its practice problem, then progress to higher levels. Focus on Medium problems first, then tackle Hard ones.